

Implemented Modules in UrbanFlow AI:

1. Backend Web Server & API

This is the central nervous system of your application, built with Flask. It orchestrates all other components.

- **Purpose:** To handle web traffic, serve the user interface, and provide data to the frontend through a series of API endpoints.
- **Key Components (app.py):**
 - **Flask Application:** The core app = Flask(__name__) instance.
 - **Routing:** Defines all API endpoints like /, /api/dashboard-data, /api/lane-feeds, and /flash/<action>.
 - **Data Aggregation:** Contains functions like get_unified_traffic_data() and get_dashboard_data() that act as a single source of truth, ensuring data consistency across the UI.
 - **Web Server:** Runs the application and makes it accessible on the network (app.run(...)).

2. AI Vehicle Detection Module

This module is the "eyes" of the system. It uses a pre-trained machine learning model to analyze video streams and identify vehicles.

- **Purpose:** To process video frames in real-time, detect and count vehicles, and provide this data to the rest of the application.
- **Key Components:**
 - **YOLOv8 Integration ([modules/detection.py](#)):** The VehicleDetector class encapsulates the Ultralytics YOLOv8 model.
 - **Real-time Processing:** The detect_vehicles() method takes a raw video frame and returns the vehicle count along with an annotated image showing bounding boxes.
 - **Data Storage:** The results are stored in the REALTIME_DATA global dictionary in [app.py](#), making vehicle counts immediately available to the data aggregation module.

3. Reinforcement Learning (DQN) Traffic Control Module

This is the "brain" of the system. It uses a Deep Q-Network (DQN) to make intelligent decisions about traffic signal timing.

- **Purpose:** To autonomously control the traffic signals to optimize traffic flow, reduce congestion, and minimize wait times.
- **Key Components:**
 - **DQN Agent ([modules/dqn.py](#)):** The TrafficDQNManager class manages the AI agent's lifecycle, including making decisions (get_action), learning from

experience (`remember_experience`, `learn_from_memory`), and saving/loading its state.

- **Control Loop (`app.py`):** The `run_dqn_control_loop` function runs in a background thread, continuously observing the environment (vehicle counts), taking actions (changing signals), and training the agent.
- **State Management:** It interacts with the `traffic_signal_backend` to execute its decisions.

4. Video Streaming & Hardware Integration Module

This module is responsible for acquiring the raw video feeds from the physical cameras and local files.

- **Purpose:** To capture video streams from the ESP32-CAM hardware and pre-recorded files and make them available to the frontend and the AI detection module.
- **Key Components (`app.py`):**
 - **ESP32-CAM Interface:** Functions like `generate_frames_on_demand()` and `generate_detection_frames()` connect to the ESP32's IP address (`ESP32_STREAM_URL`) to fetch the live MJPEG stream.
 - **Hardware Control:** The `/flash/<action>` route sends HTTP requests to the ESP32 to control its onboard LED.
 - **Video File Handling:** The system can use pre-recorded videos (`VIDEO_FILES`) for lanes, which is excellent for development, testing, and creating datasets. The `generate_frames_from_file` function handles this.
 - **Webcam Fallback:** The system is robust enough to fall back to a local webcam if the ESP32 stream fails.

5. Frontend User Interface (UI)

This is the visual and interactive part of the project that the user directly engages with.

- **Purpose:** To present the traffic data, live video feeds, and AI controls in an intuitive and accessible dashboard.
- **Key Components:**
 - **Main Dashboard (`templates/index.html`):** The single-page application structure.
 - **Styling (`static/style.css`, `static/components.css`):** Defines the look and feel, including layouts, colors, and responsive design.
 - **Client-Side Logic (`static/script.js`):** Handles user interactions (like tab switching), fetches data from the backend APIs every few seconds (`setInterval`), and dynamically updates the UI elements (charts, stats, status indicators).
 - **Traffic Simulation (`traffic_signal.js`, `traffic_signal.css`):** A dedicated sub-module for visualizing the intersection, vehicle movements, and signal states.

6. Data Logging and Analytics Module

This module is responsible for persisting the system's operational data for future analysis and model improvement.

- **Purpose:** To log every decision made by the AI agent, along with the state of the system at that time, creating a valuable dataset for performance analysis and retraining.
- **Key Components ([app.py](#)):**
 - **Dual Logging:** The system logs data to both a simple CSV file ([traffic_log.csv](#)) and a more structured SQLite database ([traffic_log.db](#)).
 - **Database Initialization:** The `initialize_database()` function creates the necessary table structure.
 - **Data Persistence:** The `append_to_log()` and `append_to_db()` functions are called at the end of each DQN control loop cycle to save a complete record of the agent's action and its outcome.
 - **Data Retrieval ([modules/insights.py](#)):** The `get_insights_data` function queries the database to calculate and return historical performance metrics for the "Insights" tab.